

EventZen

Event Management Platform

Product Requirements Document

Version 1.0	June 2025	Status: APPROVED	Classification: CONFIDENTIAL
-------------	-----------	------------------	------------------------------

Prepared by	EventZen Engineering & Product Team
Reviewed by	CTO, VP Engineering, Lead Architects
Document Type	Product Requirements Document (PRD)
Stack	React + Tailwind Spring Boot Node/Express ASP.NET Core MySQL MongoDB
Architecture	Polyglot Microservices Event-Driven Cloud-Native CI/CD Automated

Table of Contents

1. Executive Summary

- 1.1 Strategic Objectives
- 1.2 Technology Vision

2. System Architecture

- 2.1 Architecture Overview
- 2.2 Microservices Breakdown
- 2.3 Communication Patterns
 - 2.3.1 Synchronous (REST)
 - 2.3.2 Asynchronous (Apache Kafka)
- 2.4 Data Architecture — Polyglot Persistence

3. Frontend Architecture

- 3.1 Technology Stack
- 3.2 Application Structure
 - 3.2.1 Portals
 - 3.2.2 Key Pages & Routes
- 3.3 Performance & CDN Strategy

4. Backend Microservices

- 4.1 Auth & User Service (Spring Boot 3.x)
 - 4.1.1 Core Features
 - 4.1.2 API Endpoints
- 4.2 Event Management Service (Spring Boot 3.x)
 - 4.2.1 Core Features
 - 4.2.2 API Endpoints
- 4.3 Venue & Vendor Service (Node.js + Express)
 - 4.3.1 Core Features
 - 4.3.2 API Endpoints
- 4.4 Attendee & Ticketing Service (ASP.NET Core 8)
 - 4.4.1 Core Features
 - 4.4.2 API Endpoints
- 4.5 Finance & Budget Service (Spring Boot 3.x)
 - 4.5.1 Core Features
 - 4.5.2 API Endpoints

4.6 Notification Service (Node.js + Express)

- 4.6.1 Core Features
- 4.6.2 API Endpoints
- 4.6.3 Kafka Topic Subscriptions
- 4.6.4 Retry & Delivery Reliability
- 4.6.5 Template System
- 4.6.6 MongoDB Collections

5. Caching Strategy & Performance

- 5.1 Multi-Level Caching Architecture
- 5.2 Redis Caching Patterns
 - Cache-Aside Pattern (Default)
 - Write-Through Pattern (Ticket Counts)
 - Cache Key Strategy

5.3 Performance Targets

6. Robust Exception Handling

6.1 Global Error Handling Strategy

6.2 Exception Handling by Service

Spring Boot (Auth & Event & Finance Services)

Node.js/Express (Venue & Vendor, Notification Services)

ASP.NET Core (Attendee & Ticketing Service)

6.3 Custom Error Codes

6.4 Circuit Breaker & Resilience

7. DevOps, CI/CD & Infrastructure

7.1 Containerization Strategy

7.2 Docker Compose (Development)

7.3 CI/CD Pipeline (GitHub Actions)

7.4 Kubernetes Production Deployment

7.5 Observability Stack

8. Security Architecture

8.1 Authentication & Authorization

8.2 API Security

8.3 Data Security

9. Database Design (ERD Summary)

9.1 MySQL Schema — Auth & User Service

9.2 MySQL Schema — Event Management Service

9.3 MongoDB Collections — Venue & Vendor Service

9.4 MongoDB Collections — Attendee & Ticketing Service

10. Feature Module Specifications

10.1 Module Summary

10.2 Notification Service (Node.js + Express)

Supported Channels

Event Triggers

11. Non-Functional Requirements

12. Delivery Roadmap

Phase 1 — Foundation (Weeks 1–6)

Phase 2 — Core Features (Weeks 7–12)

Phase 3 — Polish & Production (Weeks 13–16)

13. Appendix

A. Technology Stack Summary

B. Document Revision History

1. Executive Summary

EventZen is a next-generation, cloud-native event management platform designed to eliminate the operational bottlenecks faced by modern event planning organizations. The platform replaces fragmented manual workflows with an integrated, real-time, microservices-based system that scales from boutique agency operations to enterprise-grade event orchestration.

This Product Requirements Document defines the complete technical blueprint, architecture, feature specifications, and delivery roadmap for the EventZen platform. It serves as the definitive reference for all engineering, product, design, and DevOps stakeholders.

Problem Statement Summary

EventZen currently struggles with: manual scheduling errors, siloed attendee tracking, opaque budget management, and no self-service customer portal. These inefficiencies result in revenue loss, client dissatisfaction, and unsustainable team overhead as event volumes grow.

1.1 Strategic Objectives

- Reduce event scheduling errors by 90% via automated conflict detection and workflow automation
- Cut attendee management overhead by 75% through self-service registration and real-time dashboards
- Achieve 100% budget traceability with real-time financial tracking and automated reporting
- Deliver sub-200ms API response times through caching, CDN, and optimized data access layers
- Support 10,000+ concurrent users via horizontally scalable microservices
- Achieve 99.9% platform uptime through redundant deployment and automated failover

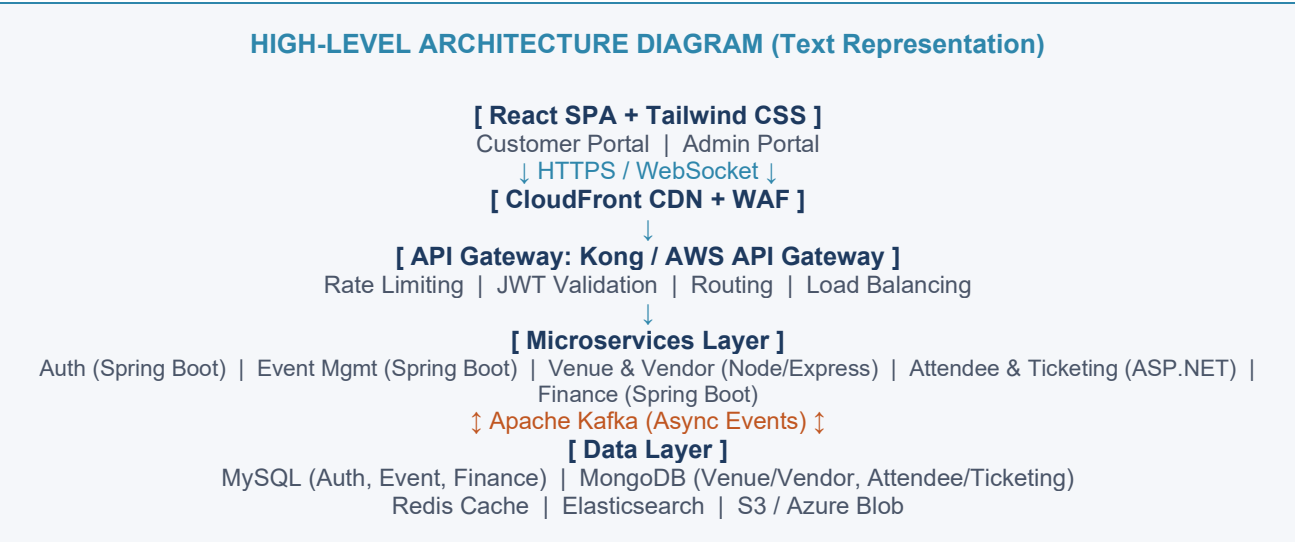
1.2 Technology Vision

The platform is built on a polyglot microservices architecture — each service independently owned, deployed, and scaled. The frontend is a React + Tailwind CSS SPA with a component-driven design system. The backend comprises four distinct microservices written in Spring Boot, Node.js/Express, and ASP.NET Core, each connected to the most appropriate database (MySQL or MongoDB). The entire platform is containerized via Docker and orchestrated with Kubernetes, with CI/CD pipelines managed through GitHub Actions.

2. System Architecture

2.1 Architecture Overview

EventZen follows a Domain-Driven Design (DDD) approach with clear bounded contexts mapped to independent microservices. Services communicate via REST APIs for synchronous operations and Apache Kafka for asynchronous, event-driven workflows. An API Gateway serves as the single entry point, handling routing, rate limiting, and authentication token validation.



2.2 Microservices Breakdown

Service	Technology	Database	Port	Responsibility
Auth & User Service	Spring Boot 3.x	MySQL 8	:8081	JWT, RBAC, Sessions
Event Management Service	Spring Boot 3.x	MySQL 8	:8082	Events, Agenda, Sessions
Venue & Vendor Service	Node.js + Express	MongoDB 7	:8083	Venues, Vendors, Contracts
Attendee & Ticketing Service	ASP.NET Core 8	MongoDB 7	:8084	Registration, Tickets, Check-in
Finance & Budget Service	Spring Boot 3.x	MySQL 8	:8085	Budget, Payments, Reports
Notification Service	Node.js + Express	MongoDB 7	:8086	Email, SMS, Push (Kafka consumer)
API Gateway	Kong / AWS API GW	Redis (config)	:8080	Routing, Auth, Rate Limit

2.3 Communication Patterns

2.3.1 Synchronous (REST)

- Client-to-Gateway: All client HTTP requests routed through API Gateway
- Gateway-to-Service: REST calls with JWT propagation and service discovery via Consul/Eureka
- Inter-service REST: Used for strong-consistency queries (e.g., Finance fetching Event details)

2.3.2 Asynchronous (Apache Kafka)

- Event topics: event.created, event.updated, event.cancelled
- Booking topics: venue.booked, ticket.purchased, registration.confirmed
- Finance topics: payment.received, budget.alert, expense.logged
- Notification topics: notification.email, notification.sms, notification.push

All Kafka messages are schema-validated via Confluent Schema Registry (Avro format) to ensure contract compliance across polyglot services.

2.4 Data Architecture — Polyglot Persistence

Service	DB Engine	Rationale	Key Collections/Tables
Auth & User	MySQL 8	ACID compliance for auth	users, roles, permissions, user_roles
Event Management	MySQL 8	Relational event data integrity	events, event_sessions, event_agenda
Venue & Vendor	MongoDB 7	Flexible schema for vendor profiles	venues, vendors, bookings, contracts
Attendee & Ticketing	MongoDB 7	High write throughput for check-in	attendees, registrations, tickets, checkins
Finance & Budget	MySQL 8	Financial ACID transactions	budgets, payments, expenses, reports
Notifications	MongoDB 7	Schema-flexible notification logs	notifications, templates, delivery_logs

3. Frontend Architecture

3.1 Technology Stack

Layer	Technology	Purpose
UI Framework	React 18 + Vite	Component-based SPA with fast HMR build tooling
Styling	Tailwind CSS 3.x	Utility-first design system, responsive layouts
State Management	Redux Toolkit + RTK Query	Global state, server cache, auto-invalidation
Routing	React Router v6	Code-split, lazy-loaded, protected routes
Forms	React Hook Form + Zod	Type-safe form validation with Zod schemas
Charts	Recharts / Chart.js	Financial dashboards, attendance analytics
Real-time	Socket.IO / WebSocket	Live check-in counters, notifications
Testing	Vitest + React Testing Library	Unit, integration, snapshot tests
E2E Testing	Playwright	Cross-browser end-to-end test automation

3.2 Application Structure

3.2.1 Portals

- Customer Portal — Event discovery, registration, ticket purchase, profile management
- Admin Portal — Full system management: events, venues, vendors, attendees, budgets, reports

3.2.2 Key Pages & Routes

Portal	Route	Description
Public	/	Landing page with event discovery
Public	/events	Paginated event listings with filters
Public	/events/:id	Event detail with registration CTA
Customer	/my/registrations	Attendee's personal booking history
Customer	/my/tickets	QR-code tickets with downloadable pass
Admin	/admin/dashboard	KPI overview, live metrics, alerts
Admin	/admin/events	Event CRUD, scheduling, agenda builder
Admin	/admin/venues	Venue inventory, hall management, bookings
Admin	/admin/vendors	Vendor contracts, service catalog
Admin	/admin/attendees	Registration management, check-in ops
Admin	/admin/finance	Budget tracking, expense management

Portal	Route	Description
Admin	/admin/reports	Analytics dashboards, export to PDF/CSV

3.3 Performance & CDN Strategy

- AWS CloudFront CDN: Static assets (JS bundles, images, fonts) served from 200+ edge locations globally
- Code Splitting: React.lazy() + Suspense for route-level code splitting; reduces initial bundle by ~60%
- Asset Optimization: Vite build pipeline applies Brotli/gzip compression, image WebP conversion, SVG optimization
- Critical Path CSS: Tailwind CSS PurgeCSS removes unused classes; production CSS < 20KB
- Service Worker: Workbox-powered PWA caching for offline-first experience on Customer Portal
- Preloading: Critical API calls prefetched via RTK Query's prefetchQuery on route hover
- Lighthouse Target: Performance > 92, Accessibility > 95, Best Practices > 95, SEO > 90

4. Backend Microservices

4.1 Auth & User Service (Spring Boot 3.x)

The Auth Service is the security backbone of the platform. It handles user lifecycle, authentication, JWT issuance, and role-based access control (RBAC). It is the only service that directly validates credentials and issues tokens.

4.1.1 Core Features

- User registration with email verification flow
- Login via JWT (Access token: 15min, Refresh token: 7 days, stored in HttpOnly cookie)
- OAuth2 Social Login: Google and GitHub
- Multi-Factor Authentication (TOTP via Google Authenticator)
- Role-Based Access Control: ADMIN, ORGANIZER, VENDOR, ATTENDEE roles
- Fine-grained Permission model: module-level actions (e.g., event:create, budget:read)
- Session management with token blacklisting via Redis on logout/revoke

4.1.2 API Endpoints

Method	Endpoint	Description	Auth Required	Role
POST	/api/v1/auth/register	Register new user	No	Public
POST	/api/v1/auth/login	Authenticate and get JWT	No	Public
POST	/api/v1/auth/refresh	Refresh access token	RefreshToken	Public
POST	/api/v1/auth/logout	Invalidate token (Redis blacklist)	JWT	Any
GET	/api/v1/auth/me	Get current user profile	JWT	Any
POST	/api/v1/auth/mfa/setup	Generate TOTP QR code	JWT	Any
POST	/api/v1/auth/mfa/verify	Verify TOTP token	JWT	Any
GET	/api/v1/users	List all users (paginated)	JWT	ADMIN
PUT	/api/v1/users/:id/roles	Assign roles to user	JWT	ADMIN
DELETE	/api/v1/users/:id	Deactivate user account	JWT	ADMIN

4.2 Event Management Service (Spring Boot 3.x)

Handles the full event lifecycle from creation to archival. Manages event categories, sessions, agendas, and integrates with the Venue service for availability checks. Publishes domain events to Kafka on every state transition.

4.2.1 Core Features

- Event CRUD with rich metadata (type, category, tags, description, banner image)
- Multi-session scheduling with speaker assignment and time-conflict detection
- Drag-and-drop agenda builder (reflected in REST API as ordered agenda items)
- Event state machine: DRAFT → PUBLISHED → REGISTRATION_OPEN → ONGOING → COMPLETED → ARCHIVED
- Automatic capacity monitoring with threshold alerts (80%, 95%, 100%)
- Recurrence rules for periodic events (weekly, monthly)
- Full-text search powered by Elasticsearch integration

4.2.2 API Endpoints

Method	Endpoint	Description	Role
POST	/api/v1/events	Create new event	ADMIN, ORGANIZER
GET	/api/v1/events	Paginated list with filters/search	Public
GET	/api/v1/events/:id	Get event details	Public
PUT	/api/v1/events/:id	Update event details	ADMIN, ORGANIZER
PATCH	/api/v1/events/:id/status	Transition event state	ADMIN
DELETE	/api/v1/events/:id	Cancel and archive event	ADMIN
POST	/api/v1/events/:id/sessions	Add session to event	ADMIN, ORGANIZER
GET	/api/v1/events/:id/agenda	Get ordered agenda	Public
PUT	/api/v1/events/:id/agenda/reorder	Reorder agenda items	ADMIN, ORGANIZER
GET	/api/v1/events/search?q={query}	Full-text search (Elasticsearch)	Public

4.3 Venue & Vendor Service (Node.js + Express)

Built on Node.js for its non-blocking I/O strengths in handling concurrent booking availability queries. MongoDB stores flexible venue configurations and rich vendor profiles. Exposes availability calendars and contract management APIs.

4.3.1 Core Features

- Venue CRUD: name, address, capacity, halls, pricing, amenities, media gallery
- Real-time availability calendar: slot-based booking conflict detection
- Vendor catalog with service categories (catering, AV, decor, security, photography)
- Vendor rating and review system
- Contract management: status tracking (PENDING → SIGNED → ACTIVE → COMPLETED)
- Venue-event linking with multi-hall support

4.3.2 API Endpoints

Method	Endpoint	Description	Role
POST	/api/v1/venues	Register new venue	ADMIN
GET	/api/v1/venues	List venues with capacity filter	Public
GET	/api/v1/venues/:id/availability	Check booking availability calendar	Authenticated
POST	/api/v1/venues/:id/book	Book venue for event	ADMIN, ORGANIZER
GET	/api/v1/vendors	Vendor catalog with service filter	Authenticated
POST	/api/v1/vendors	Register vendor	ADMIN
POST	/api/v1/events/:id/vendors	Hire vendor for event	ADMIN, ORGANIZER
PATCH	/api/v1/contracts/:id/status	Update contract status	ADMIN
POST	/api/v1/vendors/:id/reviews	Submit vendor rating & review	ADMIN, ORGANIZER

4.4 Attendee & Ticketing Service (ASP.NET Core 8)

ASP.NET Core provides the performance and concurrency needed for high-throughput event registration and real-time check-in operations. MongoDB stores attendee profiles and registration documents with fast indexed lookups.

4.4.1 Core Features

- Attendee registration with idempotency keys to prevent duplicate bookings
- Multi-tier ticketing: GENERAL, VIP, SPEAKER, SPONSOR tiers with quantity controls
- QR code generation for digital tickets (Base64 encoded, signed with HMAC-SHA256)
- Real-time check-in with QR scan validation and duplicate entry prevention
- Waitlist management with automatic promotion when capacity becomes available
- Bulk attendee import via CSV with validation pipeline
- Attendee communication: pre-event reminders, day-of instructions (via Notification Service)

4.4.2 API Endpoints

Method	Endpoint	Description	Role
POST	/api/v1/registrations	Register attendee for event	Authenticated
GET	/api/v1/events/:id/registrations	List event registrations	ADMIN, ORGANIZER
DELETE	/api/v1/registrations/:id	Cancel registration	Owner, ADMIN
GET	/api/v1/tickets/:id	Get ticket with QR code	Owner, ADMIN
POST	/api/v1/checkin/scan	Validate QR and log check-in	ADMIN, STAFF

Method	Endpoint	Description	Role
GET	/api/v1/events/:id/checkin/stats	Live check-in dashboard stats	ADMIN, ORGANIZER
GET	/api/v1/ticket-types	List available ticket tiers for event	Public
POST	/api/v1/events/:id/waitlist	Join waitlist when sold out	Authenticated
POST	/api/v1/attendees/import	Bulk CSV import attendees	ADMIN

4.5 Finance & Budget Service (Spring Boot 3.x)

The Finance Service manages the complete financial lifecycle of events. Spring Boot + MySQL ensures transactional integrity for all monetary operations. It exposes a reporting API consumed by the Admin analytics dashboard.

4.5.1 Core Features

- Budget planning: estimated, approved, and actual budget tracking
- Line-item budgeting with category breakdown (venue, catering, AV, marketing, staff)
- Payment processing integration (Stripe / Razorpay gateway abstraction layer)
- Expense logging with receipt upload (S3 storage, URL stored in DB)
- Real-time budget variance alerts: warns at 80% and 100% utilization
- Financial report generation: P&L, cash flow, expense breakdown (PDF export)
- Multi-currency support with exchange rate integration

4.5.2 API Endpoints

Method	Endpoint	Description	Role
POST	/api/v1/events/:id/budget	Create event budget plan	ADMIN, ORGANIZER
GET	/api/v1/events/:id/budget	Get budget with variance analysis	ADMIN, ORGANIZER
PUT	/api/v1/budgets/:id/approve	Approve budget	ADMIN
POST	/api/v1/budgets/:id/items	Add budget line item	ADMIN, ORGANIZER
POST	/api/v1/payments	Initiate payment (Stripe/Razorpay)	Authenticated
POST	/api/v1/payments/webhook	Payment gateway webhook handler	System (HMAC-verified)
POST	/api/v1/expenses	Log expense with receipt	ADMIN, ORGANIZER
GET	/api/v1/events/:id/reports/financial	Generate financial summary report	ADMIN
GET	/api/v1/events/:id/reports/financial/pdf	Export financial report as PDF	ADMIN

4.6 Notification Service (Node.js + Express)

The Notification Service is a dedicated, Kafka-driven microservice responsible for all outbound communications on the EventZen platform. Built on Node.js for high-throughput async I/O, it consumes domain events from Kafka topics and dispatches notifications across multiple channels: email, SMS, push, in-app WebSocket, and external webhooks. MongoDB stores notification history, Handlebars-powered templates, delivery logs, user preferences, and webhook subscriptions.

Design Principle: Event-Driven, Not API-Driven

The Notification Service has no inbound REST API for triggering notifications from other services. All triggering is exclusively via Kafka topic consumption. This decouples notification logic from business logic — producers (Auth, Event, Finance, Attendee services) simply publish domain events; the Notification Service reacts independently. This prevents tight coupling, allows retry without re-triggering business logic, and enables notification changes without touching producer services.

4.6.1 Core Features

- Multi-channel dispatch: Email (AWS SES / SendGrid), SMS (Twilio / AWS SNS), Push (FCM), In-App (Socket.IO WebSocket), Webhooks (HMAC-signed HTTP callbacks)
- Handlebars HTML template engine: customizable templates stored in MongoDB with variable interpolation and version history
- User notification preferences: per-channel, per-event-type opt-in/opt-out stored in MongoDB; respected at dispatch time
- Idempotent processing: each Kafka message carries a unique correlation ID; duplicate deliveries deduplicated via delivery_logs check
- Retry with exponential backoff: failed deliveries retried per channel-specific policy; dead-letter queue (DLQ) after max retries exceeded
- Delivery tracking: every dispatch attempt logged with provider response code, timestamp, and success/failure status
- Real-time in-app notifications: Socket.IO rooms per user; stored in MongoDB for offline users and delivered on reconnect
- Webhook subscriptions: external systems can register HTTP endpoints; payloads signed with per-subscription HMAC-SHA256 secret
- Admin notification dashboard: delivery success rates, failure heatmap by channel, template usage analytics
- Bulk broadcast: admin-triggered announcements to all registrants of an event via Kafka batch message

4.6.2 API Endpoints

While notification dispatch is exclusively event-driven via Kafka, the service exposes a REST API for notification history, template management, user preferences, and admin observability.

Method	Endpoint	Description	Auth / Role
POST	/api/v1/notifications/send	Direct trigger for system-initiated alerts (internal only)	Service mTLS
GET	/api/v1/notifications	List notification history for a user (paginated)	JWT / Any

Method	Endpoint	Description	Auth / Role
GET	/api/v1/notifications/:id	Get single notification detail and delivery status	JWT / Owner
PATCH	/api/v1/notifications/:id/read	Mark in-app notification as read	JWT / Owner
DELETE	/api/v1/notifications/:id	Delete notification from user inbox	JWT / Owner
GET	/api/v1/notifications/templates	List all notification templates	JWT / ADMIN
POST	/api/v1/notifications/templates	Create a new notification template	JWT / ADMIN
PUT	/api/v1/notifications/templates/:id	Update template content and variables	JWT / ADMIN
GET	/api/v1/notifications/delivery-logs	Admin view of all delivery attempts and statuses	JWT / ADMIN
POST	/api/v1/notifications/preferences	Update user notification channel preferences	JWT / Any
GET	/api/v1/notifications/preferences	Get user notification channel preferences	JWT / Any

4.6.3 Kafka Topic Subscriptions

The service subscribes to the following Kafka topics using a dedicated consumer group (eventzen-notification-service). All messages conform to Avro schemas registered in Confluent Schema Registry.

Kafka Topic	Channel(s)	Notification Triggered
user.registered	Email	Welcome email with verification link and onboarding steps
user.password.reset	Email	Password reset link with 15-minute expiry token
registration.confirmed	Email, SMS	Booking confirmation with QR-code ticket PDF attachment
registration.cancelled	Email	Cancellation confirmation with refund timeline
event.published	Email, Push	New event announcement to subscribed attendees
event.reminder.24h	Email, SMS, Push	Event reminder 24 hours before start with venue details
event.reminder.1h	SMS, Push	Final 1-hour reminder with check-in gate and QR code
event.cancelled	Email, SMS, Push	Event cancellation alert to all registered attendees
event.updated	Email, Push	Material change alert (venue, time, speaker) to registrants
waitlist.promoted	Email, SMS	Waitlist slot available with 2-hour registration window
payment.received	Email	Payment receipt with transaction ID and amount

Kafka Topic	Channel(s)	Notification Triggered
payment.failed	Email, SMS	Payment failure alert with retry link
budget.alert.threshold	Email, In-App	Budget utilization alert at 80% and 100% thresholds to admin
vendor.contract.signed	Email	Contract countersigned confirmation to vendor and organizer
checkin.milestone	In-App	Admin alert at 50%, 75%, 90% attendee check-in milestones

4.6.4 Retry & Delivery Reliability

Each notification channel has an independent retry policy managed via Bull (Redis-backed job queue). Failed jobs are re-enqueued with exponential backoff. After the maximum retry count is exhausted, the message is moved to a Dead Letter Queue (DLQ) topic in Kafka for manual inspection and replay.

Channel	Provider	Max Retries	Retry Backoff Strategy
Email	AWS SES / SendGrid	3	Exponential: 1 min, 5 min, 30 min
SMS	Twilio / AWS SNS	3	Exponential: 30 s, 2 min, 10 min
Push (FCM)	Firebase Cloud Messaging	5	Linear: 1 min intervals; TTL 24 h on FCM
In-App (WebSocket)	Socket.IO	N/A	Stored in DB; delivered on next connection
Webhook (External)	Custom HTTP callback	5	Exponential: 1, 5, 15, 30, 60 min

- DLQ monitoring: Kafka DLQ topic eventzen.notifications.dlq monitored by admin alerts; messages replayable via admin API
- Circuit breaker per provider: if AWS SES returns 5xx, circuit opens and falls back to SendGrid automatically
- Webhook signature: X-EventZen-Signature header with HMAC-SHA256 of payload; consumers must verify before processing

4.6.5 Template System

All notification content is managed via Handlebars templates stored in the notification_templates MongoDB collection. Templates support dynamic variable injection, conditional blocks, and partials for shared components (header, footer, branding).

- Template keys follow convention: {event_type}.{channel} — e.g., registration.confirmed.email, event.reminder.24h.sms
- Variables injected at render time from the Kafka message payload merged with user profile data
- Version history: every template update appended to versions[] array; admin can roll back to any previous version
- Preview API: admin can render a template with mock data before publishing

- Localization-ready: template lookup falls back to default locale (en) if user locale template not found

4.6.6 MongoDB Collections — Notification Service

Collection	Key Indexes	Schema Notes
notifications	user_id, status, created_at	Per-notification document; TTL index auto-expires read entries after 90 days
notification_templates	template_key (unique), channel	Handlebars HTML per channel; versions[] array for full rollback history
delivery_logs	notification_id, provider, sent_at	Per-attempt record with provider response code; powers retry audit trail
notification_preferences	user_id (unique)	Per-user opt-in/out per channel and event type; merged with defaults at dispatch
webhook_subscriptions	event_type, url	External webhook registrations with HMAC signing secret per subscription

Security: Webhook HMAC Verification

Every outbound webhook POST includes the header X-EventZen-Signature: sha256={HMAC}. The HMAC is computed over the raw request body using the per-subscription secret stored encrypted in MongoDB. Receiving systems MUST verify this header before processing to prevent spoofed event injection.

5. Caching Strategy & Performance

5.1 Multi-Level Caching Architecture

EventZen implements a three-tier caching strategy to minimize latency at every layer of the stack.

Cache Layer	Technology	TTL	Cached Data
L1: Browser	HTTP Cache Headers	1h - 1 year (assets)	Static assets, API GET responses with ETag
L2: CDN	CloudFront / Cloudflare	5min - 24h	JS/CSS bundles, images, public API responses
L3: API Gateway	Kong Cache Plugin	2min	Frequently read endpoints (venue list, event list)
L4: Application	Redis 7 (Cluster mode)	Per cache key config	User sessions, JWT blacklist, event details, ticket counts
L5: DB Query	Spring Cache + Hibernate 2L	30s - 5min	Frequently queried reference data (categories, roles)

5.2 Redis Caching Patterns

Cache-Aside Pattern (Default)

- Service checks Redis before hitting DB
- On miss: fetch from DB, write to Redis with TTL, return to client
- On update/delete: invalidate affected cache keys immediately

Write-Through Pattern (Ticket Counts)

- On ticket purchase: atomically decrement Redis counter AND write to MongoDB
- Prevents overselling under concurrent load via Redis DECR atomic operation
- Redis acts as the source of truth for real-time capacity; MongoDB synced asynchronously

Cache Key Strategy

- event:{id}:details — Full event object, TTL 10min
- event:{id}:capacity — Available tickets, TTL 30s
- user:{id}:profile — User profile, TTL 5min
- venue:{id}:availability:{date} — Slot availability, TTL 2min
- finance:{eventId}:budget — Budget summary, TTL 1min

5.3 Performance Targets

Metric	Target	Cache Hit	Cache Miss
API GET (event details)	< 50ms p99	< 5ms	< 120ms

Metric	Target	Cache Hit	Cache Miss
API POST (registration)	< 200ms p99	N/A	< 200ms
QR code check-in scan	< 100ms p99	< 10ms	< 100ms
Dashboard load (Admin)	< 1.5s LCP	< 800ms	< 1.5s
Concurrent users supported	10,000+	—	—
Platform uptime (SLA)	99.9%	—	—

6. Robust Exception Handling

6.1 Global Error Handling Strategy

Every service implements a standardized error handling framework to ensure consistent error responses, meaningful error codes, and complete observability across all microservices regardless of the underlying language or runtime.

Unified Error Response Format

All services return errors in this JSON structure: { "timestamp": "2025-06-15T10:30:00Z", "status": 400, "error": "VALIDATION_ERROR", "code": "EVT-1042", "message": "Event start time must be in the future", "path": "/api/v1/events", "traceId": "abc123-xyz456", "details": [{ "field": "start_time", "issue": "Must be future date" }] }

6.2 Exception Handling by Service

Spring Boot (Auth & Event & Finance Services)

- @RestControllerAdvice with @ExceptionHandler for centralized handling
- Custom exception hierarchy: EventZenException → BusinessException, SecurityException, IntegrationException
- @Valid + Jakarta Bean Validation for request payload validation with detailed field errors
- Hibernate ConstraintViolationException caught and mapped to 400 responses
- Transactional rollback on RuntimeException; partial failure handling for batch ops

Node.js/Express (Venue & Vendor, Notification Services)

- Centralized error middleware: app.use((err, req, res, next) => ...) as last registered middleware
- express-async-errors package to catch unhandled async rejections automatically
- Joi / Zod schema validation on all request bodies with structured field errors
- MongoDB duplicate key (E11000) and CastError mapped to appropriate HTTP codes
- Unhandled promise rejections caught via process.on('unhandledRejection') with graceful shutdown

ASP.NET Core (Attendee & Ticketing Service)

- IExceptionHandler middleware (ASP.NET 8 problem details) for standardized ProblemDetails RFC 9457 responses
- FluentValidation pipeline for request model validation
- Idempotency key validation to prevent duplicate registration on network retry
- MongoDB.Driver.MongoException hierarchy mapped to HTTP problem detail responses
- Global health checks via /health/live and /health/ready with dependency status

6.3 Custom Error Codes

Code	Category	HTTP Status	Description
AUTH-1001	Authentication Error	401 Unauthorized	Invalid or expired JWT

Code	Category	HTTP Status	Description
AUTH-1002	Authorization Error	403 Forbidden	Insufficient permissions for resource
EVT-2001	Event Business Logic	409 Conflict	Venue already booked for this time slot
EVT-2002	Event Business Logic	422 Unprocessable	Event cannot transition to requested state
TKT-3001	Ticketing Error	409 Conflict	No tickets remaining (sold out)
TKT-3002	Ticketing Error	400 Bad Request	Duplicate registration (idempotency violation)
FIN-4001	Finance Error	422 Unprocessable	Expense exceeds approved budget
FIN-4002	Payment Error	402 Payment Required	Payment gateway declined transaction
SYS-9001	System Error	503 Service Unavailable	Downstream service circuit breaker open
SYS-9002	System Error	429 Too Many Requests	Rate limit exceeded at API Gateway

6.4 Circuit Breaker & Resilience

- Resilience4j (Spring Boot) and Opossum (Node.js) for circuit breaker patterns on inter-service calls
- Circuit states: CLOSED (normal) → OPEN (failing, reject calls) → HALF_OPEN (probe recovery)
- Retry policies: exponential backoff with jitter for transient failures (max 3 retries)
- Bulkhead isolation: thread pool isolation per downstream dependency prevents cascade failures
- Timeout configuration: 2s default timeout on all inter-service HTTP calls
- Fallback responses: serve stale cached data or degraded-mode responses when dependencies fail

7. DevOps, CI/CD & Infrastructure

7.1 Containerization Strategy

Every service is packaged as a minimal, production-optimized Docker image. Multi-stage builds reduce image sizes and eliminate build toolchain from runtime images. All images are immutable and tagged with Git commit SHA for traceability.

Docker Build Strategy

Spring Boot services use eclipse-temurin:21-jre-alpine base (JRE only, ~85MB). Node.js services use node:20-alpine (~50MB). ASP.NET Core uses mcr.microsoft.com/dotnet/aspnet:8.0-alpine (~80MB). All images run as non-root user (UID 1000) for security compliance.

7.2 Docker Compose (Development)

Service	Image/Runtime	Dev Notes
auth-service	Spring Boot	Spring DevTools hot-reload, JVM debug port 5005 exposed
event-service	Spring Boot	Depends on kafka, mysql-event
venue-vendor-service	Node.js	Nodemon for hot-reload, depends on mongodb
attendee-ticketing-service	ASP.NET Core	dotnet watch, depends on mongodb
finance-service	Spring Boot	Depends on mysql-finance, kafka
frontend	Node.js/Vite	Vite HMR on port 3000, proxies /api to gateway
api-gateway	Kong	Kong declarative config (deck), port 8080
mysql	MySQL 8.0	Shared dev instance; prod uses separate DB per service
mongodb	MongoDB 7	Shared dev instance with replica set for transactions
kafka + zookeeper	Confluent	Single-broker dev setup, Kafka UI on port 8090
redis	Redis 7-alpine	Standalone dev, RedisInsight UI on port 8001
elasticsearch	Elasticsearch 8	Single-node dev, Kibana on port 5601

7.3 CI/CD Pipeline (GitHub Actions)

The CI/CD pipeline is the deployment automation backbone. Every code change to any service triggers the relevant pipeline. Pipelines are defined as YAML workflows in .github/workflows/ per service with shared reusable workflows.

#	Stage	Actions
1	Trigger	PR open, push to main/develop, manual dispatch, release tag

#	Stage	Actions
2	Code Quality	ESLint (FE), Checkstyle (Java), Prettier, SonarCloud SAST scan, secret scanning
3	Unit Tests	JUnit 5 + Mockito (Spring), Jest (Node), xUnit (.NET), Vitest (React) — minimum 80% coverage gate
4	Integration Tests	Testcontainers (MySQL/MongoDB spun up per test), REST-assured API tests, Playwright E2E on PR
5	Build & Package	Docker multi-stage build, tag with git SHA + semver, push to AWS ECR / GitHub Container Registry
6	Security Scan	Trivy container vulnerability scan, OWASP dependency-check, block on CRITICAL CVEs
7	Deploy Staging	Helm chart upgrade to K8s staging namespace, smoke tests, performance benchmark (k6 load test)
8	Approval Gate	GitHub Environment protection rules: required reviewer approval for production release
9	Deploy Production	Blue-green deployment via Kubernetes: new version deployed to green, traffic shifted after health check passes
10	Post-Deploy	Slack notification, Datadog deployment marker, automated rollback trigger if error rate > 5% in 5 minutes

7.4 Kubernetes Production Deployment

- Cluster: AWS EKS (or GKE/AKS) with managed node groups
- Namespaces: eventzen-prod, eventzen-staging, eventzen-monitoring, eventzen-infra
- HPA (Horizontal Pod Autoscaler): scale out based on CPU (70%) and custom Kafka consumer lag metrics
- Resource requests/limits: all pods have defined CPU and memory limits to prevent resource starvation
- PodDisruptionBudget: ensures minimum 1 pod available per service during rolling updates
- Secrets: AWS Secrets Manager / HashiCorp Vault — never stored in K8s Secrets in plaintext
- ConfigMaps: environment-specific application configs (non-sensitive)
- Ingress: NGINX Ingress Controller with TLS termination (cert-manager + Let's Encrypt)
- Service Mesh: Istio for mutual TLS between services, traffic shaping, and distributed tracing

7.5 Observability Stack

Pillar	Technology	Details
Logs	ELK Stack / OpenSearch	Structured JSON logs, Fluentbit forwarder, Kibana dashboards, 30-day retention
Metrics	Prometheus + Grafana	JVM metrics, Node.js metrics, custom business metrics (registrations/sec), Grafana dashboards
Tracing	Jaeger / Zipkin + OpenTelemetry	Distributed trace correlation across all services, trace sampling at 10%, full capture on errors
Alerts	Alertmanager + PagerDuty	CPU/memory thresholds, error rate spikes, latency p99 breaches, pod restart storms

Pillar	Technology	Details
Uptime	UptimeRobot / Grafana Synthetic	External health check every 30s, public status page at status.eventzen.io

8. Security Architecture

8.1 Authentication & Authorization

- JWT Bearer tokens: RS256 asymmetric signing (private key stored in HSM/Vault)
- Access token TTL: 15 minutes; Refresh token TTL: 7 days (HTTP-only secure cookie)
- Token rotation: new refresh token issued on each access token refresh (token family pattern)
- RBAC enforcement at API Gateway level (route-level) AND service level (method-level)
- Spring Security for Java services, Passport.js for Node, ASP.NET Identity for .NET

8.2 API Security

- HTTPS enforced everywhere; HSTS headers with 1-year max-age
- CORS policy: strict allowlist of origins, no wildcard in production
- Rate limiting: 100 req/min per IP for public endpoints, 1000 req/min for authenticated
- Request size limits: 10MB max payload to prevent DoS
- SQL injection prevention: parameterized queries (JPA/Hibernate, Spring Data)
- NoSQL injection prevention: Mongoose sanitize, input validation before query construction
- XSS prevention: Content-Security-Policy headers, React's built-in XSS protection
- CSRF protection: SameSite=Strict on cookies, CSRF tokens on state-changing form submissions

8.3 Data Security

- Encryption at rest: AES-256 for DB storage (MySQL Transparent Data Encryption, MongoDB encrypted storage)
- Encryption in transit: TLS 1.3 for all internal and external communication
- PII handling: email and phone stored with AES-256 application-level encryption, masked in logs
- Payment data: zero PCI scope — all card data handled by Stripe/Razorpay, tokens stored only
- GDPR compliance: data deletion API (/api/v1/users/:id/gdpr/delete), consent tracking
- Audit logging: immutable audit trail for all admin actions stored in append-only log table

9. Database Design (ERD Summary)

9.1 MySQL Schema — Auth & User Service

Table	Primary Key	Key Foreign Keys	Notable Columns
users	user_id (UUID)	—	email (unique+encrypted), password_hash, is_mfa_enabled
roles	role_id (UUID)	—	role_name, description
permissions	permission_id (UUID)	—	module, action, resource_type
user_roles	id (UUID)	user_id, role_id	assigned_by, assigned_at
role_permissions	id (UUID)	role_id, permission_id	—
refresh_tokens	token_id (UUID)	user_id	token_hash, expires_at, revoked, family_id
audit_log	log_id (BIGINT)	user_id	action, resource_type, resource_id, ip_address

9.2 MySQL Schema — Event Management Service

Table	Primary Key	Key Foreign Keys	Notable Columns
events	event_id (UUID)	organizer_id, category_id	status (enum), banner_url, max_capacity, is_recurring
event_categories	category_id (UUID)	—	category_name, icon, color
event_sessions	session_id (UUID)	event_id	speaker_id, room, session_type, capacity
event_agenda	agenda_id (UUID)	event_id	sort_order, start_time, end_time, type
event_tags	id (UUID)	event_id	tag_name (full-text indexed)

9.3 MongoDB Collections — Venue & Vendor Service

Collection	Key Indexes	Schema Notes
venues	venue_id, city, geolocation	Embedded halls array; geospatial 2dsphere index on location field
venue_bookings	venue_id + date range	Compound index on (venue_id, booking_start, booking_end) for availability queries

Collection	Key Indexes	Schema Notes
vendors	vendor_id, service_type, rating	Embedded service_packages array; portfolio_urls array for media
event_vendors	event_id, vendor_id	Contract documents with embedded version history for audit trail

9.4 MongoDB Collections — Attendee & Ticketing Service

Collection	Key Indexes	Schema Notes
attendees	attendee_id, email (unique)	Hashed email index; PII fields AES-256 encrypted at app layer
registrations	event_id + attendee_id	Unique compound index prevents duplicate registrations; idempotency_key field
ticket_types	event_id, type	available_quantity managed via Redis; MongoDB stores total; reconciled nightly
checkin_logs	registration_id, checkin_time	TTL index to auto-expire old check-in logs after 2 years; gate and staff_id tracked
waitlists	event_id + position	Queue-ordered by join_time; auto-promotion triggered by Kafka event on cancellation

10. Feature Module Specifications

10.1 Module Summary

Module	Portal	Priority	Key Capabilities
Authentication & Users	Both	P0 — Must Have	Register, login, MFA, RBAC, profile management
Event Management	Admin	P0 — Must Have	Event CRUD, sessions, agenda, state machine
Venue Management	Admin	P0 — Must Have	Venue CRUD, hall mgmt, booking calendar, conflict detection
Vendor Management	Admin	P1 — Should Have	Vendor catalog, service hiring, contracts, ratings
Attendee & Ticketing	Both	P0 — Must Have	Registration, QR tickets, check-in, waitlist
Finance & Budget	Admin	P0 — Must Have	Budget planning, payments, expense tracking, reports
Notifications	Both	P1 — Should Have	Email/SMS/push via Kafka events, templates, delivery tracking
Analytics & Reports	Admin	P1 — Should Have	KPI dashboards, trend analysis, CSV/PDF export
Search & Discovery	Customer	P1 — Should Have	Full-text event search, category filters, geo proximity
Customer Self-Service	Customer	P1 — Should Have	Booking history, ticket wallet, event reminders

10.2 Notification Service (Node.js + Express)

The Notification Service is a pure Kafka consumer — it subscribes to notification.* topics and dispatches messages via configured channels. It has no direct API exposure; all triggering is event-driven.

Supported Channels

- Email: AWS SES / SendGrid with HTML template engine (Handlebars)
- SMS: Twilio / AWS SNS for ticket confirmations and reminders
- Push: Firebase Cloud Messaging for in-app notifications
- In-App: WebSocket push to connected browser clients via Socket.IO

Event Triggers

- user.registered → welcome email with verification link
- registration.confirmed → booking confirmation email + QR ticket PDF attachment
- event.reminder.24h → event reminder email and SMS 24 hours before
- event.cancelled → cancellation notification with refund timeline
- budget.alert.threshold → admin alert email when budget reaches 80%/100%

-
- waitlist.promoted → notification to waitlisted attendee that a slot opened

11. Non-Functional Requirements

Category	Requirement	Target / Specification	Measurement
Performance	API Response Time	p50 < 50ms, p95 < 150ms, p99 < 300ms	Datadog APM
Performance	Page Load (LCP)	< 1.5s on 4G mobile	Lighthouse / Web Vitals
Scalability	Concurrent Users	10,000+ concurrent; HPA scales to 50 pods per service	k6 load tests
Availability	Uptime SLA	99.9% (< 8.7 hrs downtime/year)	UptimeRobot
Availability	RTO / RPO	RTO: 15 min / RPO: 1 hour (automated failover)	DR drill tests
Security	OWASP Top 10	Zero CRITICAL/HIGH findings in production	SAST + DAST scans
Security	Penetration Testing	Annual third-party pentest + quarterly automated DAST	Pentest report
Reliability	Error Rate	< 0.1% of all API requests return 5xx	Prometheus error rate alert
Maintainability	Code Coverage	> 80% unit test coverage; > 60% integration coverage	Codecov / JaCoCo
Compliance	Data Privacy	GDPR compliant: right to erasure, data portability, consent	Privacy audit
Accessibility	WCAG 2.1	AA conformance; screen reader support, keyboard navigation	axe DevTools

12. Delivery Roadmap

Phase 1 — Foundation (Weeks 1–6)

Wk	Deliverable	Owner	Status
1-2	Monorepo setup, Docker Compose, CI pipeline scaffolding	Platform Team	In Progress
1-2	Auth Service: register, login, JWT, RBAC	Backend - Java	In Progress
2-4	Event Management Service: CRUD, state machine	Backend - Java	Planned
2-4	React app skeleton: routing, auth context, Tailwind design tokens	Frontend Team	Planned
4-6	Venue & Vendor Service (Node.js + MongoDB)	Backend - Node	Planned
4-6	Admin Portal: event list, create event, venue booking	Frontend Team	Planned

Phase 2 — Core Features (Weeks 7–12)

Wk	Deliverable	Owner	Status
7-9	Attendee & Ticketing Service (ASP.NET Core + MongoDB)	Backend - .NET	Planned
7-9	Finance & Budget Service (Spring Boot + MySQL)	Backend - Java	Planned
9-11	QR code ticketing, check-in module, Redis caching layer	Full Stack	Planned
9-11	Customer Portal: event discovery, registration, ticket wallet	Frontend Team	Planned
11-12	Kafka integration, Notification Service, end-to-end flow testing	Platform + Backend	Planned

Phase 3 — Polish & Production (Weeks 13–16)

Wk	Deliverable	Owner	Status
13-14	Analytics dashboards, report PDF export, Elasticsearch full-text	Full Stack	Planned
13-14	Kubernetes Helm charts, staging environment, k6 load testing	Platform Team	Planned
14-15	Security hardening: pentest, SAST, dependency audit, CSP headers	Security + Platform	Planned

Wk	Deliverable	Owner	Status
15-16	UAT, bug fixes, performance optimization, documentation	All Teams	Planned
16	Production go-live: blue-green deploy, monitoring runbooks, handover	All Teams	Planned

13. Appendix

A. Technology Stack Summary

Category	Technology	Version / Notes
Frontend Framework	React + Vite	React 18, Vite 5, TypeScript 5.x
Styling	Tailwind CSS	v3.x, PostCSS pipeline, shadcn/ui component library
Java Backend	Spring Boot 3.x	Java 21 LTS, Spring Security 6, Spring Data JPA, Flyway
Node.js Backend	Node.js + Express 5	Node 20 LTS, TypeScript, Mongoose 8, Joi validation
.NET Backend	ASP.NET Core 8	C# 12, Entity Framework Core 8, FluentValidation, MediatR
Relational DB	MySQL 8.0	InnoDB engine, row-level locking, TDE enabled
Document DB	MongoDB 7	Replica set (3 nodes), Transactions, Atlas or self-hosted
Cache	Redis 7	Cluster mode (3 shards x 2 replicas), Redis Streams
Message Broker	Apache Kafka	Confluent Platform, Schema Registry, 3-broker cluster
Search	Elasticsearch 8	Full-text event search, geospatial queries, 3-node cluster
API Gateway	Kong Gateway 3.x	Declarative config, plugins: rate-limit, JWT, CORS, logging
CDN	AWS CloudFront	Edge caching, Lambda@Edge for auth, WAF rules
Container Runtime	Docker + Kubernetes	Docker 24+, K8s 1.29+, AWS EKS or self-managed
CI/CD	GitHub Actions	Reusable workflows, environments, OIDC for AWS auth
IaC	Terraform + Helm	Terraform for cloud infra, Helm 3 charts for K8s deployments
Monitoring	Prometheus + Grafana	kube-prometheus-stack, custom EventZen dashboards
Logging	ELK Stack	Elasticsearch + Logstash + Kibana, Fluentbit DaemonSet
Tracing	OpenTelemetry + Jaeger	Auto-instrumentation for all services, 10% sampling rate
File Storage	AWS S3 / Azure Blob	Event banners, expense receipts, ticket PDFs

B. Document Revision History

Version	Date	Author	Changes
1.0	June 2025	EventZen Engineering & Product Team	Initial PRD — full platform specification

EventZen Platform — Product Requirements Document

Version 1.0 | Confidential | EventZen Engineering

React + Tailwind | Spring Boot | Node/Express | ASP.NET Core | MySQL | MongoDB